

Quantum-Inspired MRFO and DMO for Cloud Task Scheduling

Results summary · makespan scheduling of independent tasks on heterogeneous VMs · all runs on an ordinary CPU (no quantum hardware)

What was done. DMO and MRFO are continuous swarm algorithms; for scheduling they are normally rounded to VM numbers, which makes them little better than random search. I replaced the rounding by a quantum-inspired representation: each task keeps a probability for every VM, a schedule is “measured” from these probabilities, and a small decoherence noise stops the search from getting stuck (Table 1). I then added a swap move for the busiest VM (Table 2) and adapted the method to a changing cloud where moving tasks has a cost (Table 3).

Key results. Quantum-inspired encoding: average gap MRFO 47.9 % → 4.3 % and DMO 59.4 % → 7.5 %. With the swap move, QI-MRFO beats the Genetic Algorithm on **76 of 80** unseen problems (0.54 % vs 2.86 %). In a changing cloud it beats the GA in **60 of 60** cases, and with migration costs it beats the best heuristic in **57–59 of 60** cases at every cost level.

How to read the tables. Gap = how much later the schedule finishes than the ideal (a theoretical lower bound); lower is better. Test problems in Tables 2 and 3 were never used for tuning.

Table 1 · Quantum-inspired encoding for DMO and MRFO

Algorithm	30 tasks 5 VMs	100 tasks 10 VMs	200 tasks 20 VMs	Average 7 problems
DMO (original, rounding)	2.89 %	75.39 %	246.16 %	59.37 %
QI-DMO (quantum-inspired)	1.65 %	3.96 %	9.02 %	7.46 %
MRFO (original, rounding)	6.52 %	83.94 %	150.00 %	47.95 %
QI-MRFO (quantum-inspired)	1.06 %	1.12 %	2.79 %	4.26 %
Genetic Algorithm (GA)	0.87 %	1.44 %	3.25 %	4.54 %
PSO	3.01 %	33.84 %	78.16 %	21.65 %
Max-Min heuristic	2.10 %	0.83 %	0.82 %	2.62 %
Random search	6.76 %	95.34 %	160.41 %	49.56 %

Setup: V4 study, 7 problems × 30 runs, 20 000 evaluations per run. QI-MRFO is at the level of the GA (4.3 % vs 4.5 %). DMO gains less because one of its steps moves randomly even when the result gets worse; with that step made greedy, QI-DMO comes close to QI-MRFO (1.46 % vs 1.12 % at 100 tasks). Source: results/baseline_full.csv, results/ablation_full.csv.

Table 2 · Adding a swap move for the busiest VM

Algorithm	Average gap	QI-MRFO + swap move better on
MRFO (original, rounding)	56.09 %	78 of 80 problems
QI-MRFO (quantum-inspired, no swap move)	3.40 %	76 of 80 problems
Genetic Algorithm (GA)	2.86 %	76 of 80 problems
GA + the same swap move	2.05 %	73 of 80 problems
Max-Min heuristic	0.63 %	67 of 80 problems
QI-MRFO + swap move (ours)	0.54 %	—

Setup: V5 held-out test, 80 unseen problems (8 problem types × 10), 2 runs each. The gap is measured against a tighter lower bound than in Table 1, so the two tables are not directly comparable. Source: results/h5_test/records.csv (analysis: results/h5_analysis.md).

Table 3 · Changing cloud: tasks come and go, VMs fail or are added

Against the GA (both with the swap move, 60 test cases): gap after each change 0.47 % for QI-MRFO vs 4.48 % for GA; QI-MRFO better in **60 of 60** cases (results/h6_dynamic).

With a cost for moving running tasks: cost gap (makespan gap plus the migration cost), 60 test cases per cost level. Best heuristic = the cheaper of full Max-Min recompute and zero-migration repair.

Migration cost	Best heuristic	Final QI-MRFO (ours)	QI-MRFO better in
Low ($\lambda = 0.05$)	3.52 %	2.25 %	58 of 60 cases
Medium ($\lambda = 0.2$)	9.19 %	5.57 %	57 of 60 cases
High ($\lambda = 1.0$)	33.90 %	15.56 %	59 of 60 cases

Source: results/h13_event_gamma/records.csv (analysis: results/h13_analysis.md).

Honest notes

- Runs on a normal CPU. A classical-probability version of the same method (no quantum formula) did as well (0.50 % in Table 2's setting), so the gain comes from the representation; no quantum advantage is claimed.
- For a one-time batch, a simple (1+1)-EA using the same swap move does as well or slightly better (better on 58 of 80 fresh problems); the swarm's own strength is the changing cloud, especially when VMs are added.
- The swap move does not carry over to QI-DMO. Added to every QI-DMO phase it made QI-DMO worse on 65 of 80 unseen problems (H14); added only where QI-DMO keeps improvements it was better on 47 of 80, but not significantly ($p = 0.067$; H15). The every-phase version does help at 500–5 000 tasks.
- All problems are generated (synthetic). Next: real cloud traces (e.g. Google, Alibaba) and energy / SLA costs.
- Method: unseen test problems, predictions written before each experiment, Wilcoxon tests with Holm correction, 269 automated tests; all numbers in this PDF are computed from the committed result files.